Trabajo Práctico

Consumo de APIs con React + TypeScript

Pokédex App — Props, Componentes y Fetch

Materia: Programación III | **Institución:** UTN TUPaD

Tecnologías: React • TypeScript • Tailwind CSS • PokéAPI

Objetivos del Trabajo Práctico

Al finalizar esta actividad, el alumno será capaz de:

- **Consumir una API REST pública** usando fetch con async/await desde un componente React.
- **Definir interfaces TypeScript** para modelar los datos que devuelve una API.
- **Diseñar una jerarquía de componentes** con al menos tres niveles de profundidad.
- **Pasar props tipadas** entre componentes padre e hijo, respetando las interfaces definidas.
- **Aplicar estilos con Tailwind CSS** usando clases utilitarias.
- **Manejar estados de carga y error** en componentes que consumen datos asíncronos.

Descripción de la Actividad

Van a construir una **Pokédex** que consume datos reales de la **PokéAPI**, una API REST pública, gratuita y sin autenticación. La aplicación muestra una grilla de Pokémon y, al hacer clic en uno, despliega su información detallada.

La app debe cumplir con los siguientes requisitos funcionales:

- **Mostrar una grilla** con al menos 20 Pokémon al cargar la página.
- **Cada tarjeta** debe mostrar la imagen del Pokémon, su nombre y sus tipos.
- **Al seleccionar un Pokémon** se deben mostrar sus estadísticas base (HP, Ataque, Defensa, etc.).
- **Mientras los datos cargan** debe mostrarse algún indicador visual (texto, spinner o skeleton).
- **Si el fetch falla** debe mostrarse un mensaje de error claro al usuario.

La PokéAPI — Endpoints y Estructura de Datos

La API base es **https://pokeapi.co/api/v2**. No requiere registro ni API key. Todos los endpoints devuelven JSON.

Endpoint 1 — Lista de Pokémon

GET `https://pokeapi.co/api/v2/pokemon?limit=20`

Devuelve una lista paginada de Pokémon con nombre y URL de detalle. Estructura de respuesta:

```
{
  "count": 1302,
  "next": "https://pokeapi.co/api/v2/pokemon?offset=20&limit=20",
  "previous": null,
  "results": [
    {
      "name": "bulbasaur",
      "url": "https://pokeapi.co/api/v2/pokemon/1/"
    },
    {
      "name": "ivysaur",
      "url": "https://pokeapi.co/api/v2/pokemon/2/"
    },
    ...
  ]
}
```

Importante

Este endpoint NO incluye imagen ni tipos. Solo nombre y URL. Para obtener la imagen y los tipos hay que hacer un segundo fetch al endpoint de detalle por cada Pokémon.

Endpoint 2 — Detalle de un Pokémon

GET `https://pokeapi.co/api/v2/pokemon/{nombre-o-id}`

Ejemplo: `https://pokeapi.co/api/v2/pokemon/pikachu` o `https://pokeapi.co/api/v2/pokemon/25`

Devuelve información completa. A continuación se muestra solo la parte relevante para este TP:

```
{
  "id": 25,
  "name": "pikachu",
  "height": 4,
  "weight": 60,
  "base_experience": 112,
  "types": [
    {
      "slot": 1,
      "type": {
        "name": "electric",
        "url": "https://pokeapi.co/api/v2/type/13/"
      }
    }
  ],
  "stats": [
    {
      "base_stat": 35,
      "effort": 0,
      "stat": { "name": "hp", "url": "..." }
    },
    {
      "base_stat": 55,
      "effort": 0,
      "stat": { "name": "attack", "url": "..." }
    },
    ...
  ],
  "sprites": {
    "front_default": "https://raw.githubusercontent.com/.../25.png",
    "front_shiny":   "https://raw.githubusercontent.com/.../shiny/25.png",
    "back_default":  "https://raw.githubusercontent.com/.../back/25.png"
  }
}
```

Consejo — Explorá la API antes de codear

Abrí `https://pokeapi.co/api/v2/pokemon/pikachu` en el navegador o en Postman y leé la respuesta completa. Vas a encontrar muchos más campos que los mostrados acá. Entender la estructura real de la respuesta es el primer paso para tiparlo correctamente.

Interfaces TypeScript

Las interfaces ya están definidas para ustedes. Su trabajo es entender cada una y usarlas correctamente al hacer los fetch y al tipar las props de los componentes.

Creen el archivo `src/types/pokemon.ts` con el siguiente contenido:

```
// Cada item que devuelve el endpoint de lista
export interface PokemonListItem {
  name: string;
  url:  string;
}

// Respuesta completa del endpoint de lista
export interface PokemonListResponse {
  count:   number;
  results: PokemonListItem[];
}

// Un tipo de pokémon (fire, water, grass...)
export interface PokemonType {
  slot: number;
  type: {
    name: string;
    url:  string;
  };
}

// Una estadística base (hp, attack, defense...)
export interface PokemonStat {
  base_stat: number;
  stat: {
    name: string;
  };
}

// Las imágenes del pokémon
export interface PokemonSprites {
  front_default: string | null;
  front_shiny:   string | null;
}

// Detalle completo — respuesta del endpoint /pokemon/{name}
export interface PokemonDetail {
  id:              number;
  name:            string;
  height:          number;  // en decímetros
  weight:          number;  // en hectogramos
  base_experience: number;
  types:           PokemonType[];
  stats:           PokemonStat[];
  sprites:         PokemonSprites;
}
```

¿Por qué dos interfaces distintas para lista y detalle?

El endpoint de lista devuelve objetos **muy simples** (solo nombre y URL). El detalle es un objeto **mucho más complejo** con tipos, stats e imágenes. Son dos llamadas HTTP a dos URLs distintas, y modelamos cada respuesta con su propia interfaz para que TypeScript nos ayude a no confundirlas.

Arquitectura de la Aplicación

La app debe organizarse en exactamente tres niveles de componentes. Cada uno tiene una responsabilidad específica y bien delimitada.

Nivel	Componente	Responsabilidad	¿Maneja estado?
1 — App	`App.tsx`	Fetch de la lista, estado global, maneja selección	Sí — datos, loading, error, pokémon seleccionado
2 — Lista	`PokemonList.tsx`	Itera el array y renderiza una card por pokémon	No — solo recibe props y las pasa hacia abajo
3 — Card	`PokemonCard.tsx`	Fetch del detalle, muestra imagen, tipos y stats	Sí — su propio loading y datos de detalle

Diagrama de flujo de datos

Así fluyen los datos y los eventos entre los tres componentes:

```
┌──────────────────────────────────────────────────────────────┐
│                          App.tsx                             │
│                                                              │
│   fetch → /pokemon?limit=20                                  │
│   estado: pokemons[], loading, error                         │
│                            │                                 │
│                  props hacia abajo                           │
│                       pokemons                               │
│                           ↓                                  │
│   ┌──────────────────────────────────────────────┐           │
│   │              PokemonList.tsx                 │           │
│   │                                              │           │
│   │   itera pokemons[] y por cada uno renderiza  │           │
│   │   una PokemonCard pasando name,              │           │
│   │                                  │           │           │
│   │              │                   │           │           │
│   │   ┌──────────────────────────┐   │           │           │
│   │   │         PokemonCard.tsx  │   │           │           │
│   │   │                          │   │           │           │
│   │   │  fetch → /pokemon/{name} │   │           │           │
│   │   │  muestra: imagen, tipos, stats │           │           │
│   │   │  al click → llama a onSelect(name) │       │           │
│   │   └──────────────────────────┘   │           │           │
│   └──────────────────────────────────────────────┘           │
└──────────────────────────────────────────────────────────────┘
```

App.tsx — Componente raíz

Es el cerebro de la app. Se encarga de obtener los datos de la API y distribuirlos hacia abajo.

Estado que debe manejar

Prop	Tipo TypeScript	Requerida	Descripción
`pokemons`	`PokemonListItem[]`	—	Lista de pokémon de la API
`loading`	`boolean`	—	true mientras se hace el fetch
`error`	`string | null`	—	Mensaje de error si el fetch falla
`selected`	`string | null`	—	Nombre del pokémon seleccionado

Props que recibe

Ninguna. App.tsx es el componente raíz, no recibe props de ningún componente padre.

Responsabilidades

- **Fetch al montar:** Al renderizarse por primera vez, debe hacer fetch al endpoint de lista y guardar los resultados en estado.
- **Manejo de loading:** Mostrar un indicador mientras el fetch está en curso. Dejar de mostrarlo al terminar, tanto si fue exitoso como si falló.
- **Manejo de error:** Si el fetch lanza una excepción, guardar el mensaje y mostrárselo al usuario.
- **Pasar datos hacia abajo:** Renderizar PokemonList pasándole la lista, el pokémon seleccionado y la función para cambiar la selección.

Hint — useEffect para el fetch inicial

Para ejecutar código al montar un componente (solo una vez), se usa `useEffect` con un array de dependencias vacío. Investigá cómo manejar el estado `loading` dentro de un bloque `try / catch / finally`: el `try` ejecuta el fetch, el `catch` captura errores de red, y el `finally` desactiva el loading pase lo que pase.

PokemonList.tsx — La grilla

Componente **puramente presentacional**. No hace fetch, no maneja estado propio. Solo recibe props y las convierte en JSX.

Props que recibe

Prop	Tipo TypeScript	Requerida	Descripción
`pokemons`	`PokemonListItem[]`	**Sí**	Array con los pokémon a mostrar
`selected`	`string | null`	**Sí**	Nombre del pokémon seleccionado actualmente
`onSelect`	`(name: string) => void`	**Sí**	Función a llamar cuando el usuario elige un pokémon

Responsabilidades

- **Iterar la lista:** Recorrer el array de pokémon y renderizar un componente PokemonCard por cada uno.
- **Pasar props individuales:** A cada PokemonCard debe pasarle el nombre, si está seleccionado (comparando con la prop selected), y la función onSelect.
- **Layout:** Organizar las cards en una grilla responsiva con Tailwind.

No hagas fetch acá

PokemonList no sabe nada de la API. Solo trabaja con los datos que le llegan por props. Si encontrás que necesitás hacer un fetch en este componente, algo está mal en el diseño.

Interface de props a definir

Definí la interface `PokemonListProps` justo arriba del componente, en el mismo archivo. Debe incluir las tres props de la tabla de arriba con sus tipos exactos.

PokemonCard.tsx — La tarjeta individual

Es el componente más completo. Recibe el nombre de un Pokémon por prop, **hace su propio fetch al endpoint de detalle**, y muestra la información cuando los datos están listos.

Props que recibe

Prop	Tipo TypeScript	Requerida	Descripción
`name`	`string`	**Sí**	Nombre del pokémon (ej: 'bulbasaur')
`isSelected`	`boolean`	**Sí**	true si es el pokémon seleccionado actualmente
`onSelect`	`(name: string) => void`	**Sí**	Función a llamar al hacer clic en la card

Estado interno

Prop	Tipo TypeScript	Requerida	Descripción
`detail`	`PokemonDetail | null`	—	Datos del pokémon obtenidos del fetch de detalle
`loading`	`boolean`	—	true mientras se carga el detalle

Responsabilidades

- **Fetch del detalle:** Al montar, hacer fetch a /pokemon/{name} y guardar el resultado en estado.
- **Estado de carga:** Mientras carga, mostrar un placeholder (puede ser un div con color de fondo y animate-pulse de Tailwind).
- **Mostrar imagen:** Usar sprites.front_default. Tener en cuenta que puede ser null.
- **Mostrar tipos:** Iterar el array types y mostrar el nombre de cada tipo.
- **Mostrar stats:** Solo cuando isSelected es true, mostrar el listado de stats con su base_stat.
- **Resaltar selección:** Cambiar visualmente la card cuando isSelected es true (borde, fondo, escala, etc.).
- **Notificar clic:** Al hacer clic en la card, llamar a onSelect pasando el name.

Hint — useEffect con dependencia [name]

El useEffect para el fetch necesita `[name]` en su array de dependencias. Esto garantiza que si la prop `name` cambia, React vuelva a ejecutar el efecto y se haga un nuevo fetch con el nombre actualizado. Con array vacío `[]`, el fetch solo correría al montar el componente y nunca se actualizaría.

Estructura de Archivos

Organización recomendada del proyecto:

```
src/
├── types/
│   └── pokemon.ts          ← Las interfaces (ya definidas, ver sección
anterior)
├── components/
│   ├── PokemonList.tsx     ← Nivel 2
│   └── PokemonCard.tsx     ← Nivel 3
├── App.tsx                 ← Nivel 1
└── main.tsx                ← No modificar
```

Configuración del Proyecto

Crear el proyecto

Ejecutar en la terminal:

```
npm create vite@latest pokedex -- --template react-ts
cd pokedex
npm install
npm install tailwindcss @tailwindcss/vite
```

Configurar Tailwind

En `tailwind.config.js`, el campo `content` debe incluir todos los archivos del proyecto:

```
content: ['./index.html', './src/**/*.{js,ts,jsx,tsx}'],
```

En `src/index.css`, reemplazar todo el contenido con las tres directivas de Tailwind: `npm @import "tailwindcss";`

Verificar errores de TypeScript

Antes de entregar, correr `npx tsc --noEmit` para verificar que no haya errores de tipos. La entrega no debe tener errores de TypeScript.

Criterios de Evaluación

Criterio	Puntos	Nota
Interfaces TypeScript usadas correctamente al tipar fetch y props	**15**	
Tres niveles de componentes bien separados con responsabilidades claras	**15**	
Props tipadas con interfaces en cada componente (PokemonListProps, PokemonCardProps)	**20**	
Fetch correcto en App.tsx con manejo de loading y error (try/catch/finally)	**15**	

Fetch de detalle en PokemonCard con useEffect y dependencia [name]	**15**	
Estilos con Tailwind CSS (grilla, hover, estado seleccionado visualmente diferenciado)	**10**	
Sin errores de TypeScript al correr npx tsc --noEmit	**10**	
TOTAL	**100**	

Puntos Extra (Opcionales)

- **+10 pts —** Agregar un campo de búsqueda para filtrar pokémon por nombre en el cliente (sin hacer un nuevo fetch a la API).
- **+10 pts —** Crear un componente StatBar.tsx que muestre cada estadística como una barra de progreso visual.
- **+10 pts —** Implementar paginación con botones Anterior y Siguiente que modifiquen el parámetro offset del fetch.
- **+5 pts —** Asignar colores distintos a cada tipo de Pokémon (fire = rojo, water = azul, grass = verde, etc.).

Preguntas Conceptuales

Responder en un archivo README.md en la raíz del proyecto. Las respuestas deben ser propias, con sus palabras.

1. **1.** ¿Por qué el estado de la lista de pokémon vive en App.tsx y no en PokemonList o PokemonCard?
2. **2.** ¿Qué diferencia hay entre un componente presentacional y un componente contenedor? ¿A cuál categoría pertenece cada uno de los tres componentes de este TP?
3. **3.** Si no pusieras [name] como dependencia del useEffect en PokemonCard, ¿qué problema concreto ocurriría?
4. **4.** ¿Por qué se usan dos interfaces distintas (PokemonListItem y PokemonDetail) en lugar de modelar todo con una sola?
5. **5.** ¿Qué ventaja tiene que PokemonList no sepa nada de la API? ¿Cómo facilita eso el testing o la reutilización del componente?

UTN TUPaD — Programación III

Tecnologías: React • TypeScript • Tailwind CSS • PokéAPI